

INSTITUTO
FEDERAL
Santa Catarina

Resumo de matérias

Programação Orientada a Objetos

Eduardo Cardoso Oliveira, 2025

Sumário

1 Como estudar para esta disciplina? 5

2 Ferramentas de Desenvolvimento 6
2.1 A linguagem Java . . . 6
2.2 Compilação, interpretação e o modelo híbrido do Java . . . 6
2.3 JVM, JRE e JDK . . . 6
2.4 Gradle . . . 7
2.5 Controle de versão: Git e GitHub . . . 7

3 Revisão de sintaxe da linguagem Java 8
3.1 Tipos primitivos e estruturas de controle . . . 8
3.2 A classe `String` . . . 8
3.3 Entrada de dados . . . 8

4 Introdução à Programação Orientada a Objetos 10
4.1 Paradigmas de programação . . . 10
4.2 O que é um objeto? . . . 10
4.3 Classes . . . 10
4.4 Abstração . . . 11
4.5 Encapsulamento . . . 11

5 Categorias de Métodos e Palavras Reservadas 13
5.1 A palavra reservada `this` . . . 13
5.2 Método construtor . . . 13
5.3 Método de representação: `toString()` . . . 14
5.4 Métodos de acesso (*getters* e *setters*) . . . 14
5.5 Métodos de comparação semântica: `equals()` e `hashCode()` . . . 14
5.6 Sobrecarga de métodos (*overload*) . . . 15
5.7 Modificador `static` . . . 15
5.8 Modificador `final` . . . 16

6 Modelagem Conceitual e UML 17
6.1 O que é a UML . . . 17
6.2 Diagrama de classes . . . 17
6.3 Análise de domínio . . . 17
6.4 Enumerações . . . 18

7 Associações e Coleções 19
7.1 O que é uma associação . . . 19
7.2 Multiplicidade . . . 19
7.3 O problema de associações muitos-para-muitos . . . 19
7.4 Navegabilidade . . . 19
7.5 Coleções . . . 20
7.6 Relações todo-parte . . . 20

8 Projeto do Sistema e API Collections 21
8.1 O que é o projeto do sistema . . . 21
8.2 Distribuição de responsabilidades . . . 21

8.3 A API Collections 21

9 Herança **23**
9.1 O que é herança 23
9.2 Quando usar (e quando não usar) herança 23
9.3 Herança em Java: um exemplo prático 23
9.4 Sobrescrita de métodos (*override*) 24
9.5 O modificador `protected` 24
9.6 Outras particularidades 24

10 Classes Abstratas, Interfaces e Polimorfismo **26**
10.1 Classes abstratas 26
10.2 Interfaces 26
10.3 Polimorfismo 27

11 Tipos Genéricos e Tratamento de Exceções **29**
11.1 Tipos genéricos (*generics*) 29
11.2 O que são exceções 29
11.3 Try-catch-finally 30
11.4 Lançando e criando exceções customizadas 30

12 Referências **32**

Aviso importante e Isenção de responsabilidade

1. **Finalidade do documento**
   Este documento foi elaborado com o objetivo de servir como um material de apoio e revisão para os estudos das matérias da 2ª fase de Análise e Desenvolvimento de Sistemas. Todos os conteúdos aqui presentes foram retirados de anotações pessoais e resumos, compilados para facilitar a compreensão do leitor.

2. **Fontes e Direitos Autorais**
   Os resumos e anotações aqui presentes foram desenvolvidos a partir dos materiais de sala disponibilizados pelos professores (como slides, apostilas, listas de exercícios, livros, etc), anotações de aula e pesquisas. Os direitos autorais sobre os materiais originais (aulas, livros, artigos) permanecem integralmente aos seus respectivos autores e editoras. Este documento é uma obra derivada, de caráter estudantil e sem fins lucrativos.

3. **Precisão das Informações**
   Desde já, o autor não oferece nenhuma garantia de que os conteúdos aqui contidos sejam precisos e corretos, por mais que todo esforço tenha sido feito para garantir que tais informações sejam corretas. Erros, omissões ou interpretações equivocadas podem existir. O autor não se responsabiliza por quaisquer consequências, acadêmicas ou de outra natureza, que possam surgir do uso deste material.

4. **Uso Recomendado**
   Este material **NÃO SUBSTITUI DE FORMA ALGUMA** a leitura de bibliografias recomendadas por professores, presença em aulas, consulta às fontes oficiais e, principalmente, as orientações do corpo docente. Utilize este documento como uma ferramenta complementar aos seus estudos.

5. **Vínculo Institucional**
   Este é um projeto independente e não possui qualquer vínculo oficial com o Instituto Federal de Santa Catarina Campus São José, a Coordenação do Curso de Análise e Desenvolvimento de Sistemas, ou os professores das disciplinas. Não há qualquer endosso ou revisão oficial do conteúdo por parte da instituição.

Apresentação do Material

Olá, colega! Seja bem-vindo(a) ao meu resumo de estudos! Este documento foi criado ao longo do semestre com o objetivo de centralizar e facilitar o acesso aos conteúdos que foram abordados durante as aulas da 2ª fase do curso em 2025.2 na disciplina de Programação Orientada a Objetos.

O que você vai encontrar neste documento?

- **Resumos:** Para esta disciplina trago resumo de slides, levantando pontos que considero os mais importantes.
- **Anotações e comentários:** Anotações sobre coisas que foram mais importantes e comentários sobre a matéria.
- **Ferramentas:** Também trago algumas recomendações de ferramentas para utilizar aqui, que ajudam no aprendizado e organização.
- **Documentos e bibliografias:** Quando possível, vou disponibilizar materiais, resumos, listas de exercícios, gabaritos e referências bibliográficas para auxiliar.

Como este documento está organizado?

O material foi dividido por blocos de conteúdo, seguindo a ordem das 5 unidades do plano de ensino da disciplina em 2025.2. Cada sessão é um resumo de algum bloco de conteúdo visto no semestre.

Informações adicionais e contato

Este material foi feito de aluno para aluno; sugestões e colaborações são sempre bem-vindas! Se encontrar algum erro, tiver alguma sugestão para adicionar ao documento ou tirar alguma dúvida sobre qualquer matéria, fique à vontade para entrar em contato.

- **Autor:** Eduardo Cardoso Oliveira
- **E-mail Institucional:** eduardo.co22@aluno.ifsc.edu.br
- **Github:** https://github.com/EduCardoso22

1 Como estudar para esta disciplina?

Programação Orientada a Objetos é, na minha opinião, uma das disciplinas mais "práticas" da 2ª fase: ela só faz sentido de verdade quando a gente coloca a mão no teclado. Diferente de matérias mais teóricas, aqui dá pra entender um conceito no slide e ainda assim travar completamente na hora de implementar, porque a dificuldade não está em decorar definições, está em pensar diferente – em *dividir um problema em objetos que conversam entre si*, ao invés de uma receita de bolo sequencial de instruções.

A disciplina é avaliada por provas e por dois projetos práticos, com apresentação oral das funcionalidades, então não dá pra "decorar na véspera". Algumas dicas que funcionaram bem pra mim:

- **Programe bastante, desde o início.** Os conceitos de classes, atributos e métodos parecem simples no papel, mas só "clicam" de verdade depois de um bom tanto de exercício prático.
- **Desenhe diagramas de classes antes de codificar.** Isso ajuda a organizar o pensamento e identificar problemas de modelagem antes de escrever a primeira linha de código.
- **Não pule a parte de ferramentas (Git, Gradle).** No começo pode parecer burocracia, mas forma a base de como você vai trabalhar em qualquer projeto profissional depois.
- **Releia o próprio código depois de um tempo.** Se você não consegue entender facilmente o que escreveu, provavelmente o encapsulamento e a divisão de responsabilidades entre as classes não estão bons.
- **Aproveite os exercícios de modelagem (UML) propostos em aula.** Eles são ótimos para treinar a identificação de classes, atributos e associações antes de partir para o Java.

2 Ferramentas de Desenvolvimento

Antes de começar a escrever qualquer linha de código em Java, é importante entender o ambiente de desenvolvimento por trás da linguagem e as ferramentas que vamos usar durante o semestre para gerenciar os projetos.

2.1 A linguagem Java

Java foi criada em 1995 por James Gosling, na Sun Microsystems (hoje parte da Oracle), e ainda está entre as linguagens mais usadas do mundo – aparecendo frequentemente no top 3 do ranking TIOBE. Suas principais características são:

- **Orientada a objetos:** todo o código é organizado em torno de classes e objetos.
- **Robusta:** resolve, de forma mais segura, problemas clássicos do C++ como gerenciamento manual de memória e ponteiros.
- **Independente de plataforma:** o lema "escreva uma vez, execute em qualquer lugar" (*write once, run anywhere*) é possível graças à Máquina Virtual Java (JVM).

2.2 Compilação, interpretação e o modelo híbrido do Java

Linguagens de programação podem ser, de forma simplificada, classificadas em:

- **Compiladas:** o código fonte é convertido de uma vez só em código de máquina (ex: C).
- **Interpretadas:** o código é lido e executado comando a comando, em tempo real (ex: Python).
- **Híbridas:** combinam as duas abordagens – é o caso do Java.

O Java segue um modelo híbrido: o código fonte (`.java`) é primeiro compilado pelo `javac` para um formato intermediário chamado **bytecode** (`.class`), que depois é interpretado (e otimizado em tempo real, através de compiladores *just-in-time*) pela JVM. É justamente esse bytecode intermediário que garante a portabilidade entre sistemas operacionais diferentes.

2.3 JVM, JRE e JDK

Esses três termos costumam confundir no início, mas a relação entre eles é simples: cada um é um "subconjunto" do próximo.

- **JVM (Java Virtual Machine):** a máquina virtual responsável por efetivamente executar o bytecode. É ela quem garante a portabilidade e cuida do gerenciamento de memória.
- **JRE (Java Runtime Environment):** inclui a JVM mais as bibliotecas fundamentais da linguagem. É o mínimo necessário para um usuário final *executar* um programa Java já pronto.
- **JDK (Java Development Kit):** inclui o JRE mais o compilador (`javac`) e outras ferramentas de desenvolvimento. É o pacote completo que um desenvolvedor precisa instalar.

2.4 Gradle

O Gradle é uma ferramenta de automação de builds, usada para compilar, testar, empacotar e executar projetos Java (entre outras linguagens). Ele é organizado em torno de *tasks* (tarefas), que podem depender umas das outras e ser personalizadas ou geradas por plugins.

Algumas tarefas comuns do plugin `application`, usado para projetos executáveis:

- `run`: executa a aplicação.
- `assembleDist`: gera arquivos `.zip`/`.tar` com a aplicação e um JRE embutido.
- `installDist`: gera scripts de instalação e execução.
- `jar`: empacota a aplicação em um arquivo `.jar` (Java ARchive), um compactado contendo classes e recursos.

Quando a aplicação depende de bibliotecas de terceiros e queremos gerar um `.jar` executável contendo tudo (um "fat jar"), é necessário incluir o plugin `shadow`, que adiciona tarefas como `shadowJar`.

```
dependencies {
    implementation 'com.google.zxing:core:3.5.3'
}
```

Listing 1: Exemplo de dependência no `app/build.gradle`

2.5 Controle de versão: Git e GitHub

Um Sistema de Controle de Versão (VCS) gerencia o histórico de alterações de um projeto, permitindo comparar, restaurar versões antigas e, principalmente, viabilizar o trabalho em equipe sem que um desenvolvedor sobrescreva o trabalho do outro – algo que pastas compartilhadas (Google Drive, Dropbox) não resolvem bem.

O **git** é o sistema de controle de versão distribuído mais usado atualmente, criado por Linus Torvalds. O **GitHub** é uma plataforma de hospedagem de repositórios git na nuvem.

Os comandos básicos ("de porcelana") mais usados no dia a dia são:

Comando	Função
`git init`	Inicializa um repositório no diretório atual
`git clone`	Clona um repositório remoto localmente
`git status`	Mostra o estado atual dos arquivos
`git add`	Prepara (*stage*) arquivos para o próximo commit
`git commit`	Registra as alterações preparadas no histórico
`git push`	Envia os commits locais para o repositório remoto

Um fluxo individual típico seria `git status` → implementar → `git add` → `git commit`, repetindo esse ciclo e, quando usando o GitHub, finalizando com `git push`.

Vale lembrar que nem tudo deve ser versionado – arquivos compilados, configurações pessoais da IDE e resultados de execução, por exemplo, devem ser listados em um arquivo `.gitignore` (o site gitignore.io gera templates prontos por linguagem e IDE). Também é boa prática manter um arquivo `README.md`, escrito em Markdown, documentando o conteúdo e a forma de execução do projeto.

3 Revisão de sintaxe da linguagem Java

Antes de entrar na orientação a objetos propriamente dita, vale revisar rapidamente a sintaxe básica do Java, já que boa parte dela é parecida com outras linguagens C-like.

3.1 Tipos primitivos e estruturas de controle

O Java possui 8 tipos primitivos (`byte`, `short`, `int`, `long`, `float`, `double`, `char` e `boolean`), que armazenam valores diretamente, sem a sobrecarga de um objeto. As estruturas de decisão (`if/else`, `switch`) e repetição (`for`, `while`, `do-while`) seguem a mesma lógica de outras linguagens baseadas em C.

```
int[] notas = {7, 8, 10, 6};
int soma = 0;
for (int nota : notas) {
    soma += nota;
}
System.out.println("Media: " + (soma / (double) notas.length));
```

Listing 2: Exemplo de laço `for-each` percorrendo um vetor

Uma pegadinha comum é a divisão entre inteiros: `1 / 2` resulta em `0`, pois ambos os operandos são `int`. Para obter o resultado real (`0.5`), é necessário converter (*typecasting*) um dos operandos para `double`, como em `(double) 1 / 2`.

3.2 A classe `String`

Strings em Java são objetos **imutáveis**: toda operação de concatenação (`+=`) cria, na verdade, um novo objeto na memória. Por isso, quando é necessário concatenar várias strings em laço, é mais eficiente usar a classe `StringBuilder`, que modifica o mesmo objeto internamente.

```
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 3; i++) {
    sb.append("Item ").append(i).append("; ");
}
String resultado = sb.toString();
// resultado: "Item 1; Item 2; Item 3; "
```

Listing 3: `StringBuilder` para concatenação eficiente

Outros métodos úteis de `String` incluem `length()`, `equals()`, `charAt()`, `substring()` e `split()` (que divide a string em um vetor a partir de um separador).

3.3 Entrada de dados

A forma mais comum de ler dados digitados pelo usuário é através da classe `Scanner`. Um detalhe importante: se um método como `nextInt()` ou `nextDouble()` for chamado antes de `nextLine()`, o caractere de nova linha (`\n`) deixado no buffer "vaza" para a próxima leitura de texto. A solução de contorno mais comum é adicionar uma chamada extra de `nextLine()` para consumir essa quebra de linha pendente.

```java
Scanner teclado = new Scanner(System.in);
int idade = teclado.nextInt();
teclado.nextLine(); // consome o '\n' deixado pelo nextInt()
String nome = teclado.nextLine();
```

Listing 4: Problema clássico do `Scanner`

4 Introdução à Programação Orientada a Objetos

4.1 Paradigmas de programação

Um paradigma de programação é, basicamente, uma forma diferente de estruturar a lógica de um programa. Todos resolvem os mesmos problemas, mas com abordagens bem distintas – e a tecnologia disponível em cada época acabou determinando qual paradigma fazia mais sentido usar.

- **Sequencial (imperativo):** o código é uma sequência de instruções, com uso pesado de desvios incondicionais (`GOTO`). Funciona bem para problemas pequenos, mas vira uma bagunça rapidamente em sistemas maiores. Lembra uma receita de cozinha. Ex: Assembly, Basic.
- **Funcional/estrutural:** o problema é dividido em procedimentos (funções), que são chamados de forma organizada para resolver partes específicas do problema. É mais rígido, como as engrenagens de um relógio. Ex: Pascal, C.
- **Orientado a objetos:** o sistema é visualizado como um conjunto de "células" interconectadas (os objetos), cada uma com seu próprio estado e responsabilidade, que trocam mensagens entre si para resolver o problema. É flexível e dinâmico, como células do corpo. Ideal para sistemas de médio/grande porte e alta complexidade. Ex: Java, Python.

4.2 O que é um objeto?

Um objeto é um item identificável (possui uma identidade própria), com características próprias (**estado**) e capacidade de fazer algo (**comportamento**).

- **Estado:** representa as características do objeto. Um carro, por exemplo, tem cor, modelo, velocidade atual e um dono.
- **Comportamento:** representa as ações que o objeto pode realizar (ou sofrer). O mesmo carro pode acelerar, frear, trocar marcha ou ser vendido.

Conseguir identificar estados e comportamentos de objetos do mundo real é o primeiro passo para começar a pensar em termos de orientação a objetos. Objetos também podem ter graus de complexidade bem diferentes (uma garrafa d'água vs. um computador) e podem ser compostos por outros objetos (um computador tem tela, teclado e gabinete).

Em Java, um objeto guarda seu estado em **atributos** e seu comportamento é definido por **métodos**, que podem ler ou alterar esses atributos.

4.3 Classes

Se um objeto é uma instância concreta, uma **classe** é o molde, o projeto ou a planta que define como esses objetos devem ser construídos. É como a planta de um Fusca: todo Fusca fabricado (objeto) segue o mesmo projeto (classe).

Em Java, classes são declaradas com a palavra reservada `class`, e objetos são criados (*instanciados*) usando `new`. Os membros de um objeto são acessados usando o caractere ponto (`.`).

```
public class ContaBancaria {
    // Atributos (estado)
    String titular;
    double saldo;

    // Métodos (comportamento)
    void depositar(double valor) {
        saldo += valor;
    }

    void sacar(double valor) {
        saldo -= valor;
    }
}

// Instanciando e usando o objeto
ContaBancaria conta = new ContaBancaria();
conta.titular = "Eduardo";
conta.depositar(100);
conta.sacar(30);
```

Listing 5: Uma classe simples e sua instanciação

Na notação UML, uma classe é representada por um retângulo dividido em três seções: nome, atributos e métodos, no formato `identificador : tipo`.

4.4 Abstração

Abstração é o processo mental de focar nos aspectos mais relevantes de alguma coisa, ignorando os detalhes menos importantes – é o que permite gerenciar a complexidade de um objeto, concentrando esforços nas características essenciais para o contexto do problema que estamos resolvendo.

Um ponto importante é que a abstração **depende do contexto**. O que é relevante em um sistema pode ser completamente irrelevante em outro. Por exemplo, para um sistema de revenda de veículos, uma classe `Carro` provavelmente precisa de atributos como `codigo`, `marca`, `modelo`, `ano` e `preco`. Já para um jogo de corrida, a mesma classe `Carro` precisaria de atributos completamente diferentes, como `cor`, `marchaAtual` e `velocidadeMaxima`, com métodos como `acelerar()` e `frear()`.

4.5 Encapsulamento

Encapsulamento é o processo de esconder os detalhes internos de um objeto que não contribuem para sua operação essencial, expondo apenas uma interface controlada para o mundo externo. É basicamente o mesmo princípio por trás de usar um aplicativo de transporte: você informa origem e destino e recebe um preço, mas não precisa saber (nem tem acesso) a como a rota ou o preço foram calculados internamente.

O benefício é duplo: o objeto fica protegido contra interferências externas indevidas no seu estado, e ao mesmo tempo ganha uma interface clara de como deve ser usado.

Em Java, isso é feito através dos **modificadores de acesso**:

- `public`: o membro pode ser acessado por qualquer outro objeto do sistema.

- `private`: o membro só pode ser acessado dentro da própria classe.
- *(ausência de modificador)*: conhecido como *package-private*, permite acesso a qualquer classe do mesmo pacote.

Por convenção, atributos devem ser `private` e métodos devem ser `public` (com exceções pontuais de métodos auxiliares privados). A regra geral é sempre usar o modificador mais restritivo possível, para manter a integridade do estado do objeto.

```
public class ContaBancaria {
    private double saldo; // protegido! não dá para alterar de fora

    public void depositar(double valor) {
        if (valor > 0) saldo += valor; // só altera com validação
    }
}

ContaBancaria c = new ContaBancaria();
c.saldo = 999999; // ERRO de compilação! saldo está encapsulado
c.depositar(100); // forma correta de alterar o saldo
```

Listing 6: Diferença entre encapsulado e não encapsulado

5 Categorias de Métodos e Palavras Reservadas

5.1 A palavra reservada `this`

`this` é uma referência ao próprio objeto. Seu uso é obrigatório sempre que um parâmetro de método tiver o mesmo nome de um atributo da classe, para diferenciar um do outro. Também pode ser usada para chamar outros métodos ou construtores da própria classe.

```
public class Produto {
    private String nome;

    public void setNome(String nome) { // mesmo nome do atributo
        this.nome = nome; // this.nome = atributo | nome = parâmetro
    }
}
```

Listing 7: Uso do `this` para diferenciar atributo de parâmetro

5.2 Método construtor

O construtor é um método especial, responsável por atribuir valores iniciais aos atributos durante a criação de um novo objeto. Algumas regras importantes:

- Não pode ser chamado como um método comum (não existe `objeto.Construtor()`).
- Não possui tipo de retorno.
- Tem obrigatoriamente o mesmo nome da classe.
- Se a classe não define nenhum construtor, Java gera automaticamente um construtor padrão vazio implícito.

Quando nenhum construtor é definido (ou um atributo não é inicializado), os atributos recebem valores padrão: `0` para tipos numéricos, `false` para `boolean`, `'\u0000'` para `char` e `null` para objetos.

Construtores também podem ser **sobrecarregados** (ver próxima seção), permitindo criar o objeto de diferentes formas – uma técnica comum é um construtor mais simples chamar, através de `this(...)`, um construtor mais completo, evitando repetição de código.

```
public class Produto {
    private String nome;
    private double preco;

    public Produto() {
        this("Produto sem nome", 0.0); // chama o construtor de baixo
    }

    public Produto(String nome, double preco) {
        this.nome = nome;
        this.preco = preco;
```

```
    }
}
```

Listing 8: Construtores sobrecarregados e encadeados

5.3 Método de representação: `toString()`

Todo objeto em Java possui um método `toString()` padrão herdado implicitamente, que por padrão retorna apenas o nome da classe e o endereço de memória do objeto – algo bem pouco útil na prática. Por isso é comum **sobrescrever** (*override*) esse método para que ele retorne um texto legível, representando bem o objeto.

```
public class Produto {
    private String nome;
    private double preco;
    // ... construtor omitido

    @Override
    public String toString() {
        return String.format("Produto(nome=%s, preco=%.2f)", nome, preco);
    }
}

Produto p = new Produto("Teclado", 150.0);
System.out.println(p); // Produto(nome=Teclado, preco=150.00)
```

Listing 9: Sobrescrevendo `toString()`

5.4 Métodos de acesso (*getters* e *setters*)

São os métodos que a classe oferece para que outros objetos possam ler (`get`) ou atualizar (`set`) atributos privados, seguindo o padrão de nomenclatura `getAtributo()` e `setAtributo(valor)`. É importante lembrar que *setters* devem conter alguma lógica de validação – caso contrário, eles acabam sendo a forma mais comum de "furar" o encapsulamento, já que qualquer valor arbitrário passa a ser aceito sem restrição.

5.5 Métodos de comparação semântica: `equals()` e `hashCode()`

Por padrão, dois objetos em Java só são considerados iguais quando apontam para o mesmo endereço de memória (igualdade de *identidade*). Isso nem sempre é o que queremos: duas instâncias diferentes de `Pessoa` com o mesmo CPF, por exemplo, deveriam provavelmente ser consideradas "iguais" do ponto de vista do negócio. Para isso, sobrescrevemos `equals()` (e, por consequência, também `hashCode()`, já que os dois devem ser consistentes entre si – principalmente para o correto funcionamento de estruturas como `HashSet` e `HashMap`, que veremos mais adiante).

```
public class Pessoa {
    private String cpf;
    // ... outros atributos e métodos

    @Override
```

```
    public boolean equals(Object o) {
        if (!(o instanceof Pessoa outra)) return false;
        return this.cpf.equals(outra.cpf);
    }

    @Override
    public int hashCode() {
        return Objects.hash(this.cpf);
    }
}
```

Listing 10: Comparação semântica por CPF

5.6 Sobrecarga de métodos (*overload*)

Uma classe pode ter vários métodos com o mesmo nome, desde que cada um tenha uma **assinatura** diferente (nome, quantidade, tipo e ordem dos parâmetros). Essa verificação é feita em tempo de compilação, e não durante a execução.

```
public class Calculadora {
    public int somar(int a, int b) {
        return a + b;
    }

    public double somar(double a, double b) {
        return a + b;
    }

    public int somar(int a, int b, int c) {
        return a + b + c;
    }
}
```

Listing 11: Sobrecarga de métodos

5.7 Modificador `static`

O modificador `static` indica que um membro (atributo ou método) pertence **à classe**, e não a um objeto específico. Atributos estáticos são compartilhados por todas as instâncias da classe (ótimos para contadores, por exemplo), e métodos estáticos podem ser chamados sem a necessidade de criar um objeto (como `Math.round()`). Um detalhe importante: um método estático **não pode** acessar diretamente atributos não estáticos, já que não há um objeto específico associado a ele.

```
public class Produto {
    private static int totalCriados = 0;

    public Produto() {
        totalCriados++; // compartilhado por TODOS os objetos Produto
    }
```

```
    public static int getTotalCriados() {
        return totalCriados;
    }
}
```

Listing 12: Contador compartilhado entre instâncias

5.8 Modificador `final`

O modificador `final` é usado para evitar mudanças, e seu efeito depende do contexto: em **variáveis/atributos**, torna o valor imutável após a primeira atribuição (por convenção, constantes são escritas em maiúsculo); em **métodos**, impede que sejam sobrescritos por subclasses (veremos herança já já); em **classes**, impede que a classe seja herdada. Um detalhe sutil: um objeto referenciado por uma variável `final` pode ter seus atributos internos alterados normalmente – o que não pode mudar é a *referência* em si.

6 Modelagem Conceitual e UML

6.1 O que é a UML

A UML (*Unified Modeling Language* – Linguagem de Modelagem Unificada) define notações gráficas, conjuntos semânticos e regras sintáticas para especificar, construir e documentar os artefatos de um sistema. Ela pode ser usada de duas formas: como um **rascunho** informal, para explorar partes complexas de um problema, ou como um **projeto** detalhado, usado tanto para gerar código a partir de modelos (engenharia direta) quanto para entender um código já existente (engenharia reversa).

Também existem duas perspectivas de uso importantes de se diferenciar:

- **Perspectiva conceitual:** representa o domínio do problema, focando em entidades do mundo real – é usada na análise de requisitos, sem se preocupar com detalhes técnicos de implementação.
- **Perspectiva de implementação:** representa a estrutura real do software, já próxima do código (atributos tipados, associações como referências concretas), usada no projeto detalhado e na codificação.

Algumas ferramentas úteis para desenhar diagramas: Mermaid, PlantUML, Draw.io, StarUML e Astah.

6.2 Diagrama de classes

O diagrama de classes é usado durante a concepção e elaboração de um sistema, representando o aspecto estático da informação – ou seja, sua estrutura, deixando evidente quais classes existem e como elas se relacionam. Ele é composto por três elementos principais: **classes** (nome, atributos e métodos), **associações** (vínculos entre objetos de classes diferentes, que permitem a troca de mensagens) e **marcadores** (comentários, trechos de código ou pacotes, quando necessário).

Alguns marcadores adicionais importantes na notação de uma classe: sinais de `+`/`-` para indicar visibilidade pública/privada; sublinhado para indicar membros estáticos; e a notação `{readOnly}` (ou `{r/o}`) para indicar um atributo constante (equivalente ao `final` do Java). Métodos triviais, como *getters* e *setters* básicos, costumam ser omitidos do diagrama para não poluí-lo.

6.3 Análise de domínio

A análise de domínio é a etapa que antecede a implementação, e consiste em descobrir e descrever informações do problema para a modelagem de classes. É um artefato independente de tecnologia, e normalmente envolve conversar com outras pessoas e setores.

Uma técnica simples para identificar classes e atributos é analisar uma descrição textual do problema: **substantivos** geralmente viram classes ou atributos ("livro", "pagamento"), enquanto **verbos** geralmente viram métodos ("comprar", "pagar"). O processo, de forma resumida, segue três passos: (1) selecionar as informações relevantes, (2) agrupar sinônimos e (3) classificar entre atributos e conceitos (classes).

Exemplo prático: imagine o enunciado "o sistema de uma livraria digital permite que um comprador escolha livros e os adicione ao carrinho de compras; cada livro possui título, autor e preço". Daí já é possível identificar conceitos claros como `Comprador`, `Livro` e `CarrinhoDeCompras`, e atributos como `titulo`, `autor` e `preco` (pertencentes a `Livro`).

6.4 Enumerações

Enumerações são um "meio-termo" entre atributos e classes: representam um conjunto limitado e pré-definido de opções válidas para um atributo, como os dias da semana ou os naipes de um baralho. Podem ser incluídas no diagrama, mas em seu próprio pacote, e geralmente não possuem associações.

7 Associações e Coleções

7.1 O que é uma associação

Uma associação indica que objetos de uma classe possuem um vínculo com objetos de outra classe, permitindo a troca de mensagens entre eles – é justamente o que dá vida ao diagrama de classes, além de simplesmente listar as classes isoladas. Um `Carro` está associado a um `Motor`, por exemplo, pois um objeto `Carro` contém um objeto `Motor`.

Para evitar ambiguidades (já que duas classes podem se relacionar de mais de uma forma), é comum acrescentar um **nome de papel** à associação. Quando o vínculo é de um para um, esse nome de papel também pode ser usado diretamente como identificador do atributo que representa essa associação no código.

7.2 Multiplicidade

A multiplicidade define quantas instâncias de uma classe podem estar associadas a uma instância de outra classe.

Notação	Significado
`1`	Exatamente uma
`0..1`	Zero ou uma (associação opcional)
`*`	De zero a infinito
`1..*`	Uma ou mais
`2,4`	Duas ou quatro

Quando a multiplicidade inclui `0`, a associação é opcional. Quando a multiplicidade do outro lado é maior do que `1`, a associação passa a representar uma **coleção** de objetos (próxima subseção).

7.3 O problema de associações muitos-para-muitos

Quando a associação entre duas classes é de muitos-para-muitos, é comum existir informação pertinente à própria associação, e não a nenhuma das duas classes isoladamente. Por exemplo: o instrumento tocado por um artista em uma música específica, ou a quantidade de itens de um produto em um pedido específico. Nesses casos, a solução é criar uma nova classe para representar essa informação:

- **Classe de associação:** usada quando o par de objetos das classes associadas deve ser único. Seu identificador é o próprio par de objetos, e ela é representada "pendurada" na linha de associação.
- **Classe de transação:** usada quando o par *não* precisa ser único (ex: um mesmo artista pode tocar um mesmo instrumento em várias músicas diferentes, em situações diferentes). Seu identificador é independente, e ela é inserida *entre* as duas classes originais, com multiplicidade múltipla internamente e unitária nas pontas externas.

7.4 Navegabilidade

A navegabilidade indica a direção em que uma associação pode ser percorrida, ou seja, qual classe efetivamente tem acesso à outra. Ter acesso implica manter uma referência

ao(s) outro(s) objeto(s) armazenada no próprio estado – é, portanto, um aspecto mais ligado à implementação do que à perspectiva conceitual.

- **Bidirecional:** ambas as classes podem mandar mensagens uma para a outra; é necessário manter a referência nos dois lados. No diagrama, uma linha simples já indica esse comportamento por padrão.
- **Unidirecional:** apenas uma classe conhece a outra. No diagrama, uma seta aponta na direção da associação.

7.5 Coleções

Uma coleção é um objeto que agrupa e gerencia um conjunto de outros objetos, sendo representada como uma associação com multiplicidade maior que um no diagrama de classes. Coleções diferentes têm comportamentos diferentes quanto à repetição de elementos e à ordenação:

Categoria	**Repetição**	**Ordenação**	**Exemplo conceitual**
Conjunto (*set*)	Não	Não	Uma frota de veículos
Multiconjunto (*bag*)	Sim	Não	Visualizações de um vídeo
Conjunto ordenado	Não	Sim	Tabela de classificação
Lista (*list*)	Sim	Sim	Playlist de músicas
Pilha (*stack*, LIFO)	Sim	Sim	Pilha de pratos
Fila (*queue*, FIFO)	Sim	Sim	Fila de atendimento

Pilhas seguem a política LIFO (*last-in, first-out* – o último elemento a entrar é o primeiro a sair), enquanto filas seguem a política FIFO (*first-in, first-out* – o primeiro a entrar é o primeiro a sair).

7.6 Relações todo-parte

Uma relação todo-parte é um tipo especial de associação em que a completude de um objeto se torna dependente do acesso a outro. Existem três níveis de "força" nessa relação:

- **Dependência (mais fraca):** o relacionamento é momentâneo, sem vínculo estrutural permanente. No diagrama, é representada por uma seta tracejada. Ex: um jogador *requer* um dado para jogar uma partida.
- **Agregação (forte, mas não exclusiva):** ocorre quando uma classe é um "contêiner" de outra, mas o ciclo de vida da parte não depende do todo. Representada por um losango *vazado*. Ex: um carro requer quatro rodas para ser dirigido, mas as rodas podem existir (e ser reaproveitadas) independentemente do carro específico.
- **Composição (forte e exclusiva):** a existência da parte depende diretamente do todo – o ciclo de vida de ambos está vinculado. Representada por um losango *preenchido*. Ex: um livro é composto por seus capítulos; se o livro deixa de existir, os capítulos também deixam.

8 Projeto do Sistema e API Collections

8.1 O que é o projeto do sistema

Enquanto a análise de domínio identifica *o que* existe no problema, o projeto do sistema define *como* os objetos interagem entre si para realizar as operações desejadas – ou seja, qual objeto faz o quê, e como. Ele se divide em duas camadas: o **projeto lógico** (detalha classes e interações na perspectiva de implementação, como o diagrama de classes completo do projeto) e o **projeto tecnológico** (trata da infraestrutura e tecnologias utilizadas).

8.2 Distribuição de responsabilidades

Uma das decisões de projeto mais importantes é: *quais métodos devem ficar em cada classe?* O risco de não pensar nisso com cuidado é concentrar funções demais em poucas classes (geralmente uma classe "controladora" central), deixando as demais passivas e o projeto mal distribuído – o que vai contra os princípios da orientação a objetos.

Dois princípios de projeto ajudam a guiar essa distribuição:

- **Princípio da Responsabilidade Única (SRP):** cada classe deve se responsabilizar por uma única tarefa, mantendo-se coesa e fácil de manter.
- **Princípio da Separação de Preocupações (SoC):** separar as preocupações do sistema em módulos independentes, facilitando desenvolvimento e testes em paralelo.

De forma prática, as responsabilidades de uma classe podem ser classificadas em dois grandes grupos, cada um dividido em três subgrupos (sobre si mesma, sobre seus vizinhos, e outras):

	Conhecer (consultas)	**Fazer** (operações)
Sobre si	`getAtributo()`	`setAtributo(valor)`
Sobre vizinhos	`getPapel()`	`addPapel()` / `removePapel()`
Outros	Conhecimentos derivados	Ações coordenadas

Conhecimentos derivados são valores calculados a partir de outros atributos (ex: um `Circulo` com atributo `raio` pode ter um método derivado `getArea()`). **Ações coordenadas** (ou métodos delegados) são operações que alteram múltiplos objetos de uma vez, e devem ser implementadas na classe que tiver o melhor acesso (mais direto) aos objetos envolvidos – por exemplo, um método `marcarTodosComoLidos()` faz mais sentido estar na classe que possui a coleção de itens do que espalhado pelo sistema.

8.3 A API Collections

A API Collections do Java é um conjunto de interfaces e classes que permite armazenar, organizar e manipular grupos de objetos de forma eficiente e reutilizável, evitando que cada desenvolvedor precise reimplementar estruturas de dados básicas (listas encadeadas, tabelas hash, árvores balanceadas) do zero. Ela é organizada em torno de três componentes: **interfaces** (contratos, como `List`, `Set`, `Map`), **estruturas de dados** (os mecanismos internos: vetor, lista encadeada, tabela hash, árvore) e **implementações concretas** que combinam uma interface com uma estrutura.

Interface	Características	Implementações comuns
`List`	Permite duplicatas, mantém ordem de inserção	`ArrayList`, `LinkedList`
`Set`	Não permite duplicatas	`HashSet`, `TreeSet`, `LinkedHashSe`
`Map`	Mapeia chaves únicas para valores	`HashMap`, `TreeMap`
`Queue/Deque`	Geralmente segue ordem FIFO	`LinkedList`, `ArrayDeque`

```
List<String> tarefas = new ArrayList<>();
tarefas.add("Estudar POO");
tarefas.add("Fazer o resumo");
tarefas.add("Estudar POO"); // duplicata permitida

System.out.println(tarefas.get(0)); // Estudar POO
System.out.println(tarefas.size()); // 3

// Removendo com expressão lambda
tarefas.removeIf(t -> t.equals("Fazer o resumo"));

for (String tarefa : tarefas) {
    System.out.println(tarefa);
}
```

Listing 13: Usando a interface `List` com `ArrayList`

Algumas boas práticas importantes ao usar coleções:

- Coleções só aceitam objetos, nunca tipos primitivos diretamente – por isso usamos `Set<Integer>` e não `Set<int>` (o Java faz a conversão automática entre `int` e `Integer`, chamada *autoboxing*).
- **Programe para interfaces, não para implementações**: prefira declarar a variável como `List<Pessoa> lista = new ArrayList<>();` em vez de `ArrayList<Pessoa> lista = new ArrayList<>();`. Isso deixa o código mais flexível para trocar a implementação no futuro, se necessário.
- Para percorrer um `Set`, nunca use índices (eles nem existem na interface) – use `for-each` ou `forEach()` com uma expressão lambda.

9 Herança

9.1 O que é herança

Herança é uma associação especial de hierarquia entre duas classes: uma classe mais genérica (**superclasse**) e outra mais específica (**subclasse**), que *especializa* o comportamento da primeira. A superclasse *generaliza* o comportamento da subclasse, tornando-o mais amplo. Também pode ser interpretada, de forma resumida, como uma relação do tipo "é-um".

O termo vem, inclusive, da biologia: na classificação taxonômica, diferentes espécies são agrupadas em categorias cada vez mais amplas (gênero, família, ordem...), e todas as espécies de um mesmo grupo compartilham características em comum. Em POO funciona de forma parecida: um `Gato` *é um* `Mamífero`, que *é um* `Animal`.

9.2 Quando usar (e quando não usar) herança

Herança é indicada para **generalizar** (fatorar) informações comuns de várias classes em uma só, reduzindo repetição e aumentando reutilização, e para **especializar** uma classe quando é necessário um comportamento adicional ou diferente, mantendo a relação "é-um" verdadeira durante todo o ciclo de vida do objeto (ou seja, o objeto nasce e morre do mesmo tipo – não há "transição" de tipo).

Usada de forma inadequada, a herança pode reduzir a flexibilidade do código, criar dependências rígidas entre classes e dificultar a manutenção. Alguns casos clássicos de **abuso** da herança, que merecem atenção:

- Quando a superclasse não generaliza nenhuma característica comum real das subclasses.
- Quando as subclasses são estruturalmente idênticas (nesse caso, uma simples enumeração resolveria melhor).
- Quando a "subclasse" só existe em função de uma terceira classe do sistema – por exemplo, modelar `Aluno` como subclasse de `Pessoa` só porque a pessoa está matriculada em uma `Escola`. Nesse caso, "Aluno" é, na verdade, um **papel** que a pessoa assume em uma associação com `Escola`, e o correto seria resolver isso com uma classe de associação ou com composição, não com herança – usar herança aqui aumenta a fragilidade do sistema e pode duplicar registros.

9.3 Herança em Java: um exemplo prático

Imagine um sistema de pagamentos, em que um pedido pode ser pago por cartão, PIX ou boleto. Todas as formas de pagamento compartilham `valor` e `data`, mas cada uma tem seus próprios dados específicos (número do cartão, chave PIX, código de barras). Modelar tudo em uma única classe forçaria atributos desnecessários em cada tipo de pagamento; duplicar tudo em classes separadas geraria repetição de código. A solução elegante é a herança:

```
public class Pagamento {
    protected double valor; // note o modificador protected
    protected LocalDate data;
```

```
    public Pagamento(double valor, LocalDate data) {
        this.valor = valor;
        this.data = data;
    }

    public String confirmar() {
        return String.format("Pagamento de R$ %.2f confirmado", valor);
    }
}
```

Listing 14: Superclasse `Pagamento`

```
public class PagamentoPix extends Pagamento {
    private String chavePix;

    public PagamentoPix(double valor, LocalDate data, String chavePix) {
        super(valor, data); // chama o construtor da superclasse
        this.chavePix = chavePix;
    }

    @Override
    public String confirmar() {
        return String.format("Pagamento via PIX (%s) confirmado", chavePix);
    }
}
```

Listing 15: Subclasse especializando o comportamento

9.4 Sobrescrita de métodos (*override*)

Métodos da superclasse podem ser sobrescritos pelas subclasses (já vimos isso acontecendo naturalmente com `toString()` e `hashCode()`), permitindo que cada subclasse especifique seu próprio comportamento enquanto mantém o "contrato" de realizar aquele método. É boa prática (e o IntelliJ cobra isso) marcar esses métodos com a anotação `@Override`, o que ajuda o compilador a detectar erros de assinatura.

9.5 O modificador `protected`

Além de `public`, `private` e *package-private*, a herança introduz o modificador `protected`: os membros protegidos podem ser acessados pela própria classe, por outras classes do mesmo pacote, **e também pelas suas classes derivadas** (mesmo que estejam em um pacote diferente). É um meio-termo entre `private` e `public`, pensado justamente para a herança – no exemplo acima, `valor` e `data` são `protected` para que as subclasses de pagamento consigam acessá-los diretamente.

9.6 Outras particularidades

- A palavra-chave `super` dá acesso aos membros da superclasse: `super()` chama o construtor da superclasse; `super.metodo()` chama a versão de um método definida na superclasse (mesmo que tenha sido sobrescrito); `super.atributo` acessa o valor do atributo antes da especialização da subclasse.

- Em Java, **toda classe herda implicitamente de `Object`** – é lá que estão definidos os comportamentos padrão de métodos como `hashCode()` e `toString()`.
- Java **não permite herança múltipla** de classes (uma subclasse com várias superclasses), o que evita problemas conceituais complexos de "herança múltipla de estado". Veremos na próxima seção como as interfaces contornam essa limitação.
- O operador `instanceof` permite testar, em tempo de execução, se um objeto é de um determinado tipo (considerando toda a hierarquia de herança): `carro instanceof Veiculo` retorna `true` se `Carro` for (direta ou indiretamente) subclasse de `Veiculo`.

10 Classes Abstratas, Interfaces e Polimorfismo

10.1 Classes abstratas

Uma classe abstrata pode conter atributos, métodos concretos (com implementação) e métodos **abstratos** (sem implementação, apenas a assinatura) – mas, diferente de uma classe comum, **nunca pode ser instanciada** diretamente. Todo método abstrato é de implementação obrigatória pelas subclasses concretas; se uma subclasse não sobrescrever um método abstrato herdado, ela também precisa ser declarada como abstrata.

O principal motivo para usar uma classe abstrata é garantir, em tempo de compilação, que todas as subclasses concretas implementem determinado comportamento – diferente de uma superclasse comum, que apenas sugere (mas não obriga) a sobrescrita de um método.

```
public abstract class FormaGeometrica {
    public abstract double calcularArea();
    public abstract String desenhar();
}

public class Circulo extends FormaGeometrica {
    private double raio;

    public Circulo(double raio) { this.raio = raio; }

    @Override
    public double calcularArea() {
        return Math.PI * raio * raio;
    }

    @Override
    public String desenhar() {
        return String.format("Desenhando circulo de raio %.2f", raio);
    }
}

// FormaGeometrica f = new FormaGeometrica(); // ERRO! Não pode instanciar
Circulo c = new Circulo(5.0); // ok, classe concreta
```

Listing 16: Classe abstrata para um aplicativo de desenho vetorial

10.2 Interfaces

Uma interface é semelhante a uma classe abstrata "pura": define um contrato que deve ser implementado por uma classe, mas, diferente de uma classe abstrata, não é propriamente uma classe. Em uma interface:

- Todos os atributos são implicitamente `public static final` (ou seja, constantes que pertencem à interface, não às classes que a implementam).
- Todos os métodos são implicitamente `public abstract` por padrão – mas métodos `static` e `default` podem ter implementação.

A grande vantagem das interfaces é que, embora Java não permita herança múltipla de classes, **uma classe pode implementar várias interfaces ao mesmo tempo**, viabilizando uma forma segura de "herança múltipla de tipos" (contratos), sem os problemas de herdar atributos (estado) de múltiplas fontes.

```
public interface Voador {
    String voar();
}

public interface Nadador {
    String nadar();
}

// Um pato pode voar E nadar - herança múltipla de tipos
public class Pato implements Voador, Nadador {
    @Override
    public String voar() { return "O pato esta voando"; }

    @Override
    public String nadar() { return "O pato esta nadando"; }
}
```

Listing 17: Uma classe implementando múltiplas interfaces

10.3 Polimorfismo

Polimorfismo significa, literalmente, "muitas formas". É a capacidade de um objeto ser referenciado (tratado) de diferentes formas – mais especificamente, permite que um objeto de uma classe específica seja tratado como um objeto de uma classe mais genérica da sua hierarquia (uma superclasse ou uma interface que ele implemente).

A grande vantagem prática é a facilidade para criar sistemas extensíveis: novas classes podem ser adicionadas ao sistema exigindo pouca ou nenhuma modificação nas partes mais gerais do código – desde que façam parte de uma hierarquia já existente. A regra de ouro é: **programe pensando na classe mais genérica, não na mais específica**.

O exemplo clássico é guardar objetos de subclasses diferentes em uma mesma coleção, tipada pela superclasse (ou interface) em comum, e percorrer essa coleção chamando um método que cada subclasse implementa de forma diferente:

```
List<FormaGeometrica> formas = new ArrayList<>();
formas.add(new Circulo(5.0));
formas.add(new Quadrado(4.0));
formas.add(new Triangulo(3.0, 6.0));

// Mesmo código funciona para qualquer tipo de forma, presente ou futura
for (FormaGeometrica forma : formas) {
    // Em tempo de execucao, édecidido qual implementacao será chamada
    System.out.println(forma.desenhar() + " - area: " + forma.calcularArea());
}
```

Listing 18: Polimorfismo com uma lista de formas geométricas

Se amanhã criarmos uma nova classe `Hexagono` que estenda `FormaGeometrica`, o laço acima **não precisa ser alterado** – basta adicionar o novo objeto à lista. É importante notar, porém, que o contrário não é verdadeiro: tudo o que é possível fazer com uma `FormaGeometrica` é possível fazer com um `Circulo`, mas um método específico de `Circulo` (que não existe na superclasse) não pode ser chamado através de uma referência do tipo `FormaGeometrica` sem antes fazer um *casting* explícito.

11 Tipos Genéricos e Tratamento de Exceções

11.1 Tipos genéricos (*generics*)

O problema que os tipos genéricos resolvem é simples de visualizar: imagine uma classe `Caixa` capaz de armazenar um valor de qualquer tipo, usando internamente o tipo `Object` (que é superclasse de tudo em Java). O problema é que, ao recuperar o valor, é necessário fazer um *casting* explícito – e, se o tipo estiver errado, o erro só vai aparecer **em tempo de execução**, o que é bem pior do que detectá-lo durante a compilação.

Tipos genéricos resolvem isso permitindo que tipos (classes ou interfaces) sejam passados como parâmetro durante a definição de uma classe, interface ou método – de forma parecida com como um parâmetro normal recebe um *valor*, um parâmetro de tipo genérico recebe um *tipo*.

```
public class Caixa<T> {
    private T conteudo;

    public void guardar(T item) {
        this.conteudo = item;
    }

    public T retirar() {
        return this.conteudo;
    }
}

Caixa<String> caixaDeTexto = new Caixa<>();
caixaDeTexto.guardar("Ola mundo");
String texto = caixaDeTexto.retirar(); // sem necessidade de casting!

// Caixa<String> c2 = new Caixa<>();
// c2.guardar(42); // ERRO DE COMPILACAO, e nao de execucao
```

Listing 19: Classe `Caixa` genérica

É possível ainda restringir quais tipos são aceitos com `extends` (ex: `Caixa<T extends Animal>` só aceitaria tipos que sejam `Animal` ou subclasses dele). Por convenção, nomes de parâmetros de tipo costumam ser uma única letra maiúscula: `T` para tipos, `E` para elementos (usado amplamente na própria API Collections), `K` para chaves (*key*), `V` para valores (*value*) e `N` para números.

11.2 O que são exceções

Uma exceção é um evento que ocorre durante a execução de um programa e interrompe o fluxo normal das instruções – em Java, exceções são representadas por objetos. Exemplos comuns incluem divisão por zero, acesso a um índice inválido de um vetor, tentativa de abrir um arquivo inexistente ou uma conversão de tipo inválida.

Toda exceção deriva de `Throwable`, que se divide em dois ramos principais: `Error` (erros graves, que geralmente não devem ser capturados/tratados pela aplicação) e `Exception` (exceções que podem, e devem, ser tratadas). Dentro de `Exception`, existe uma distinção importante:

- **Checked exceptions:** devem ser obrigatoriamente tratadas ou declaradas na assinatura do método (com `throws`), e são verificadas já em tempo de compilação. Ex: `IOException`, `SQLException`.
- **Unchecked exceptions** (subclasses de `RuntimeException`): não precisam ser tratadas explicitamente, e geralmente representam erros de programação. Ex: `NullPointerExcept` `ArrayIndexOutOfBoundsException`.

11.3 Try-catch-finally

O bloco `try` contém o código que pode gerar uma exceção; o bloco `catch` captura e trata a exceção; e o bloco `finally` (opcional) é executado sempre, com ou sem exceção – útil para garantir a liberação de recursos.

```
public static void dividir(int a, int b) {
    try {
        int resultado = a / b;
        System.out.println("Resultado: " + resultado);
    } catch (ArithmeticException e) {
        System.out.println("Erro: nao e possivel dividir por zero!");
    } finally {
        System.out.println("Operacao de divisao finalizada.");
    }
}
```

Listing 20: Tratamento básico de exceção

É possível ter múltiplos blocos `catch`, do mais específico para o mais genérico – a ordem importa, pois o primeiro `catch` compatível é o que será executado. Desde o Java 7, também existe o `try-with-resources`, que fecha automaticamente recursos que implementem `AutoCloseable` (como arquivos), sendo mais seguro e conciso do que usar `finally` manualmente para isso.

11.4 Lançando e criando exceções customizadas

A palavra-chave `throw` lança uma exceção manualmente, e `throws` na assinatura de um método declara que ele *pode* lançar determinada exceção. Também é possível (e, em sistemas maiores, recomendável) criar exceções customizadas, estendendo `Exception` (para uma checked exception) ou `RuntimeException` (para uma unchecked), com nomes descritivos do problema que representam.

```
public class SaldoInsuficienteException extends Exception {
    public SaldoInsuficienteException(double saldo, double valorSolicitado) {
        super("Saldo insuficiente: possui R$ " + saldo +
              ", tentou sacar R$ " + valorSolicitado);
    }
}

public class ContaBancaria {
    private double saldo;

    public void sacar(double valor) throws SaldoInsuficienteException {
```

```
        if (valor > saldo) {
            throw new SaldoInsuficienteException(saldo, valor);
        }
        saldo -= valor;
    }
}
```

Listing 21: Exceção customizada para saldo insuficiente

Algumas boas práticas no tratamento de exceções: seja específico (capture o tipo exato de exceção esperado, não apenas `Exception`); nunca ignore uma exceção silenciosamente; documente exceções lançadas usando `@throws` no Javadoc; "falhe rápido", lançando a exceção assim que o problema for detectado; e forneça mensagens de erro claras e descritivas.

12 Referências

Referências

[1] GOOGLE DRIVE. Programação Orientada a Objetos. Disponível em: `https://drive.google.com/drive/folders/1zcvfTSnQqmXXI7rMkw1-dLhf2zeAjmF2?usp=drive_link`. Acesso em: 22 de outubro de 2025.

[2] IFSC - CAMPUS SÃO JOSÉ. Plano de Ensino: Programação Orientada a Objetos, 25.2. Prof. Sergio Prolo. Acesso em: 22 de outubro de 2025.

[3] HORSTMANN, Cay S.; CORNELL, Gary. *Core Java Volume I: Fundamentos*. 8. ed. Pearson, 2010.

[4] BEZERRA, Eduardo. *Princípios de análise e projeto de sistemas com UML*. 3. ed. Campus, 2002.

[5] DEITEL, P. J.; DEITEL, H. M. *Java: como programar*. 8. ed. São Paulo: Pearson Prentice Hall, 2010.

[6] LARMAN, C.; BRAGA, R. V. *Utilizando UML e padrões: uma introdução à análise e ao projeto orientados a objetos e ao desenvolvimento iterativo*. 3. ed. Porto Alegre: Bookman, 2007.

[7] SCHILDT, H. *Java para iniciantes: crie, compile e execute programas Java rapidamente*. 6. ed. Porto Alegre: Bookman, 2015.

[8] CAELUM. Apostila FJ-11 Java e Orientação a Objetos. Disponível em: `https://www.caelum.com.br/apostila/apostila-java-orientacao-objetos.pdf`. Acesso em: 22 de outubro de 2025.

[9] GOOGLE. Google Java Style Guide. Disponível em: `https://google.github.io/styleguide/javaguide.html`. Acesso em: 22 de outubro de 2025.

[10] ORACLE. The Java Collections Framework. Disponível em: `https://docs.oracle.com/en/java/javase/21/core/java-collections-framework.html`. Acesso em: 22 de outubro de 2025.